

WEEK 6: EXERCISE 6

Database System and Cloud (BERR2243)

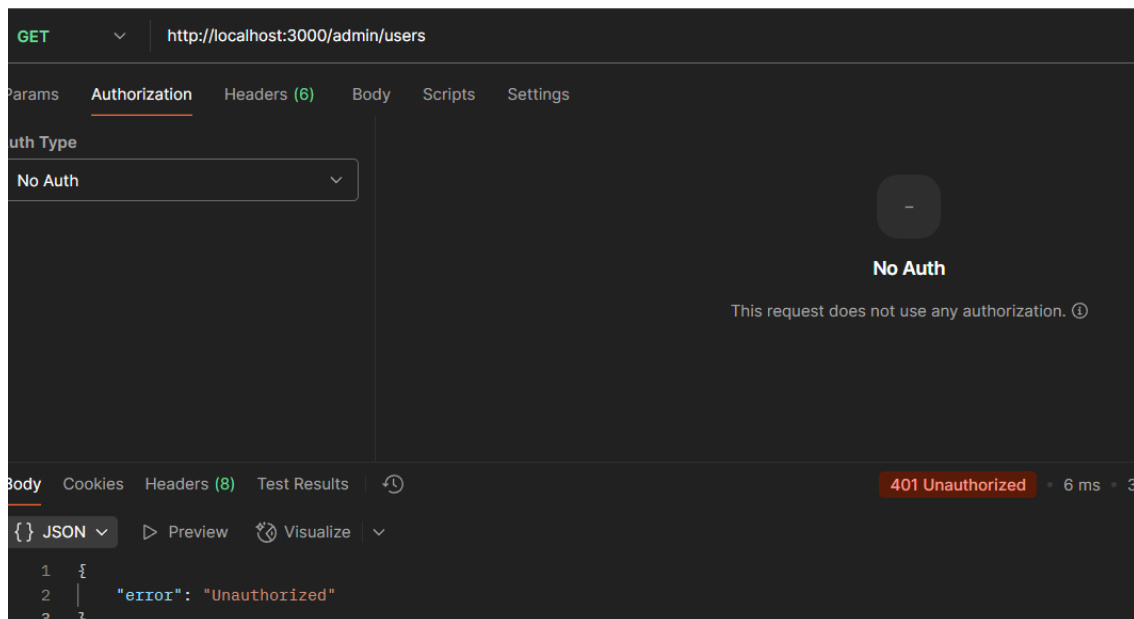


Provided by:

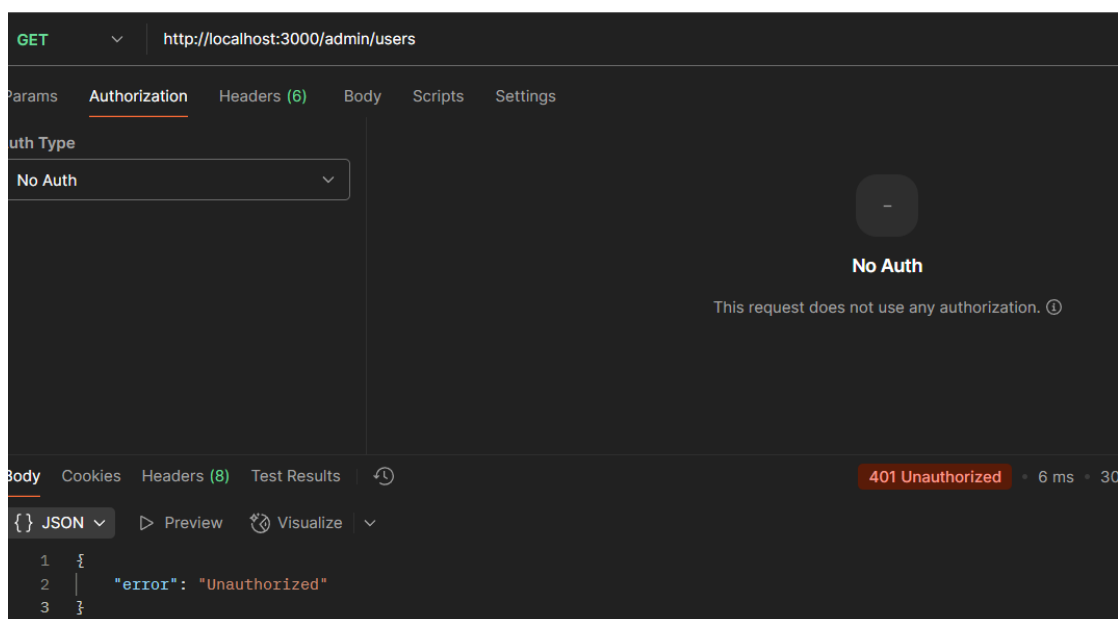
Group Member	Id No.
Ahmad Muhaimin Azeem Bin Adri Wafik	B122310496, 2 BERR S2
Syafiq Iman Bin Suhaimi	B122310218, 2 BERR S2
Ery Fahrhan Bin Mohamed Ali	B122310448, 2 BERR S2

1. Token Usage:

o What happens if you omit the Authorization header when accessing `/admin/users/{id}`?



o What error occurs if you use an expired token?



o What happens if the rides collection is empty?

Paste the token generated to <https://jwt.io>, and discuss the content

[Get an exclusive look at jwt.io v2 and help us shape its final form with your feedback. →](#)

HEADER:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD:

```
{
  "userId": "6840f5b6d9f3ad4deb037c5d",
  "role": "driver",
  "iat": 1749087694,
  "exp": 1749091294
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
)
```

☐ secret base64 encoded

Header: It specifies the signing algorithm (HS256) and token type (JWT).

Payload: Contains the claims:

userId: The unique ID of the user.

role: The role of the user (driver in this case).

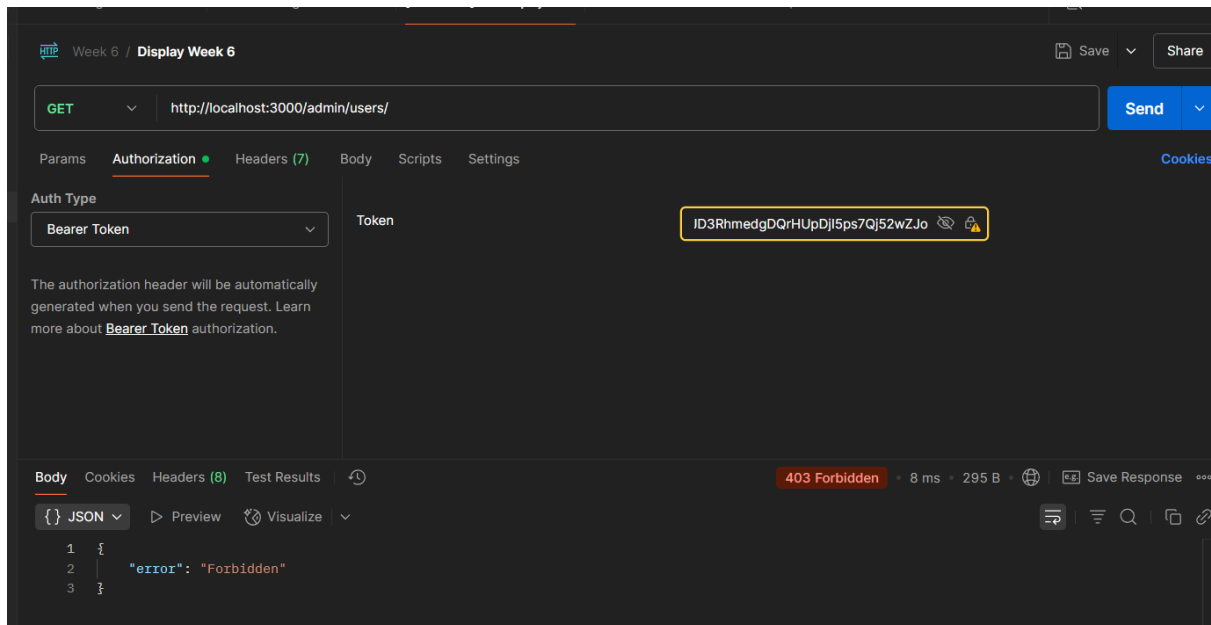
iat: The issued-at timestamp, showing when the token was created.

exp: The expiration timestamp, showing when the token will expire.

Signature: Ensures the token's integrity by verifying it hasn't been altered.

2. Role Restrictions:

o If a **customer-role user** tries to access `/admin/users/{id}`, what status code is returned?



o How would you modify the middleware to allow both admin and driver roles to access an endpoint?

```
// GET /admin/users - Show All Users Info (protected: admin only)
app.get('/admin/users', authenticate, authorize(['admin','driver']), async (req, res) => {
  try {
    const users = await db.collection('users').find().toArray();
    res.status(200).json(users);
  } catch (err) {
    res.status(500).json({ error: "Failed to fetch users" });
  }
});

// GET /admin/drivers - Show All Drivers Info (protected: admin only)
app.get('/admin/drivers', authenticate, authorize(['admin','driver']), async (req, res) => {
  try {
    const drivers = await db.collection('drivers').find().toArray();
    res.status(200).json(drivers);
  } catch (err) {
    res.status(500).json({ error: "Failed to fetch drivers" });
  }
});

// DELETE /admin/users/:id - Block users (protected: admin only)
app.delete('/admin/users/:id', authenticate, authorize(['admin','driver']), async (req, res) => {
  try {
    const result = await db.collection('users').deleteOne({ _id: new ObjectId(req.params.id) });
    if (result.deletedCount === 0) return res.status(404).json({ error: "User Not Found" });
    res.status(204).send(); // No Content
  } catch (err) {
    res.status(400).json({ error: "Invalid User Id Or Data" });
  }
});
```

3. Security:

o Why is the JWT token sent in the Authorization header instead of the request body?

The Authorization header is more secure than placing the token in the request body or URL parameters because HTTP headers are less likely to be logged or exposed in transit compared to body data or URL query strings.

o How does password hashing protect user data in a breach?

Password hashing makes passwords unrecoverable in the event of a compromise, hence protecting user data. It turns plaintext passwords into hashed values, which are one-way, irreversible, and frequently salted for further uniqueness. Even if hashed passwords are taken during a breach, they cannot be easily reversed to reveal the original password, making it far more difficult for attackers to get access to user accounts.

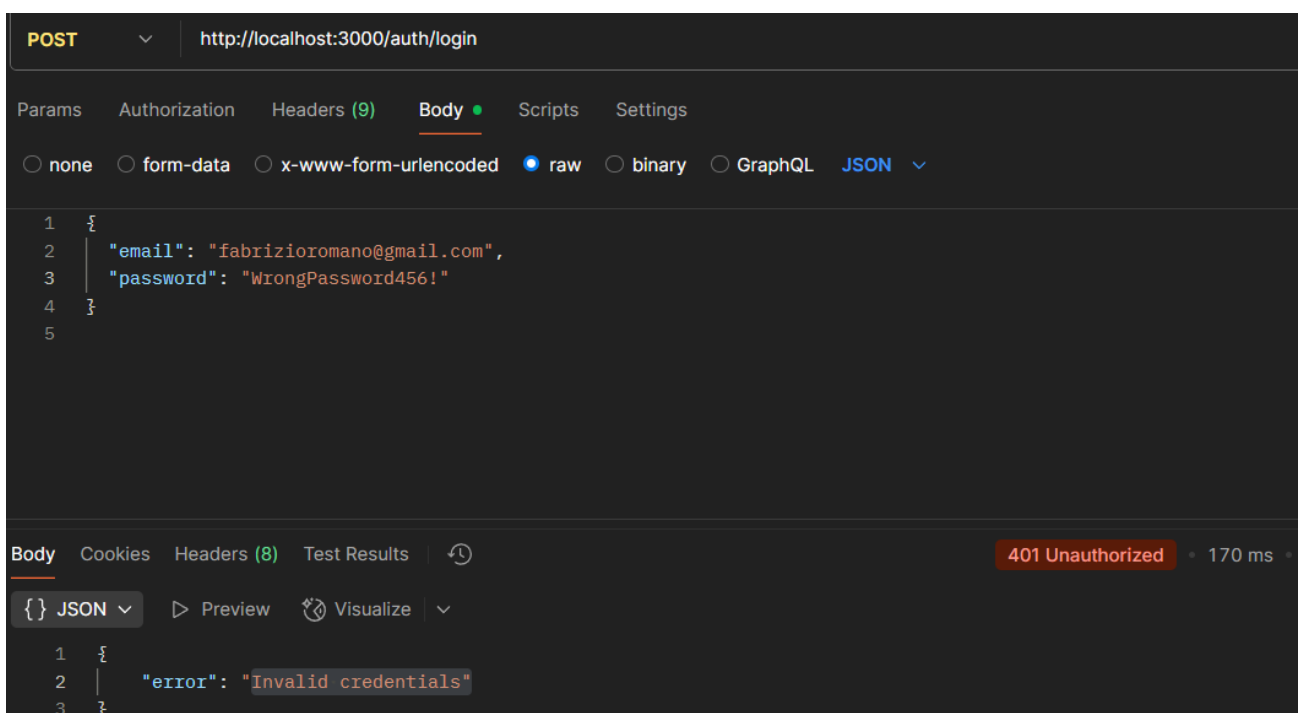
4. Postman Testing:

o What is the purpose of the Bearer keyword in the Authorization header?

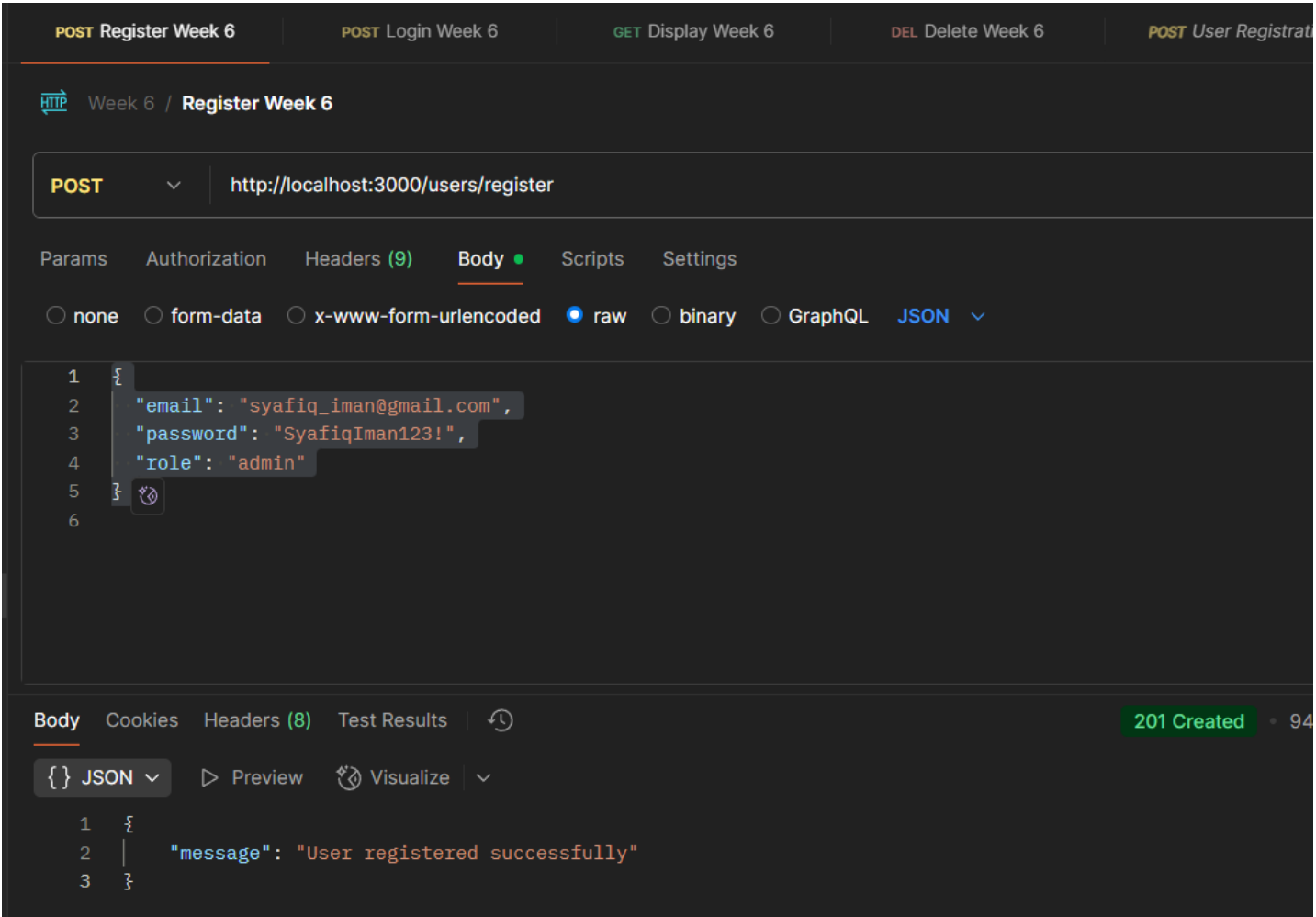
The "Bearer" keyword in the Authorization header is part of the OAuth 2.0 standard and is used to indicate that the token being sent is a bearer token.

A bearer token is a type of access token that allows the bearer (whoever holds the token) to access protected resources without needing further authentication.

o How would you test a scenario where a user enters an incorrect password?



SCREENSHOTS FOR POSTMAN



Successful Registration

POST Register Week 6 | **POST Login Week 6** | GET Display Week 6 | DEL Delete Week 6 | POST User Registration | + | No environment

Week 6 / **Login Week 6** | Save | Share

POST | http://localhost:3000/auth/login | Send

Params | Authorization | Headers (9) | **Body** | Scripts | Settings | Cookies | Beautify

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☒ JSON

```
1 {
2   "email": "syafiq_iman@gmail.com",
3   "password": "SyafiqIman123!"
4 }
5
```

Body | Cookies | Headers (8) | Test Results | 200 OK • 75 ms • 476 B | Save Response

{ } JSON | Preview | Visualize

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2ODQyNTRjNjM4MzA1YmIwMDI1Y2EzNDQ1LCJyb2x1IjoieWRTaW4iLCJpYXQiOiJlE3NDkxNzc3NTcsImV4cCI6MTc0TE4MTM1N30.T2hosA7CivTx-PDfuft8TpQotFmuQ_0zahMBLcYj4kE"
3 }
```

Successful Log In

HTTP Week 6 / Display Week 6

GET http://localhost:3000/admin/users/

Params Authorization Headers (7) Body Scripts Settings

Auth Type: Bearer Token

Token: [REDACTED]

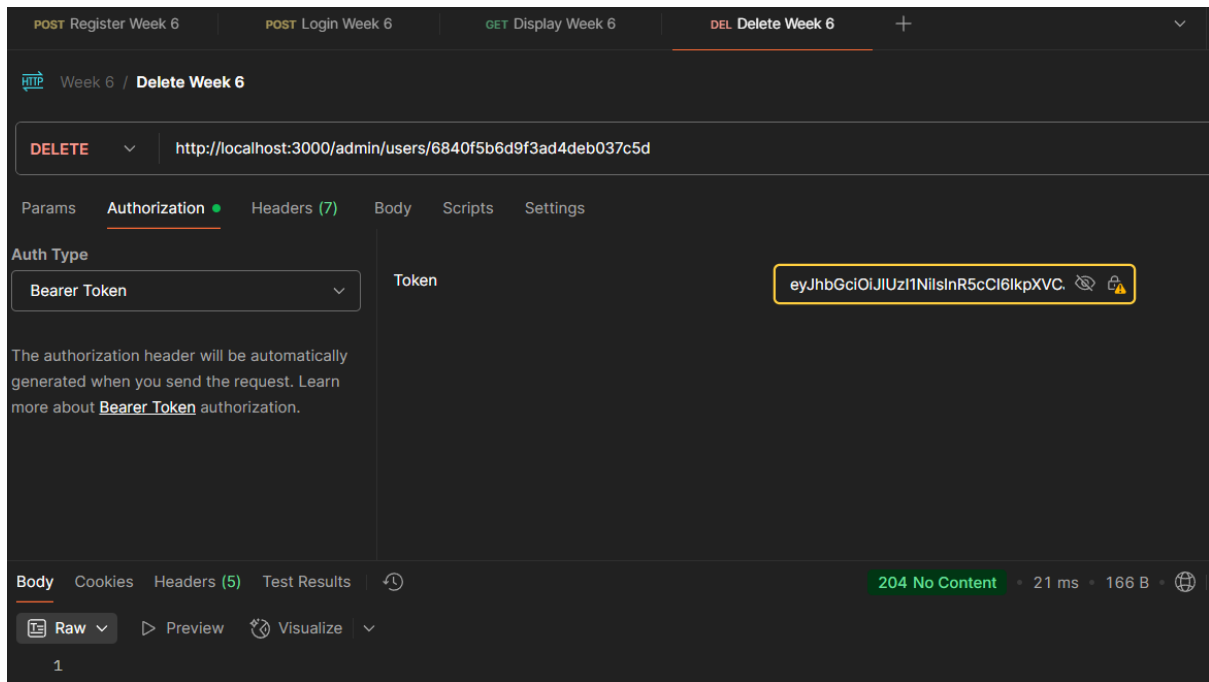
The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies Headers (8) Test Results 200 OK • 8 ms • 2.41 KB

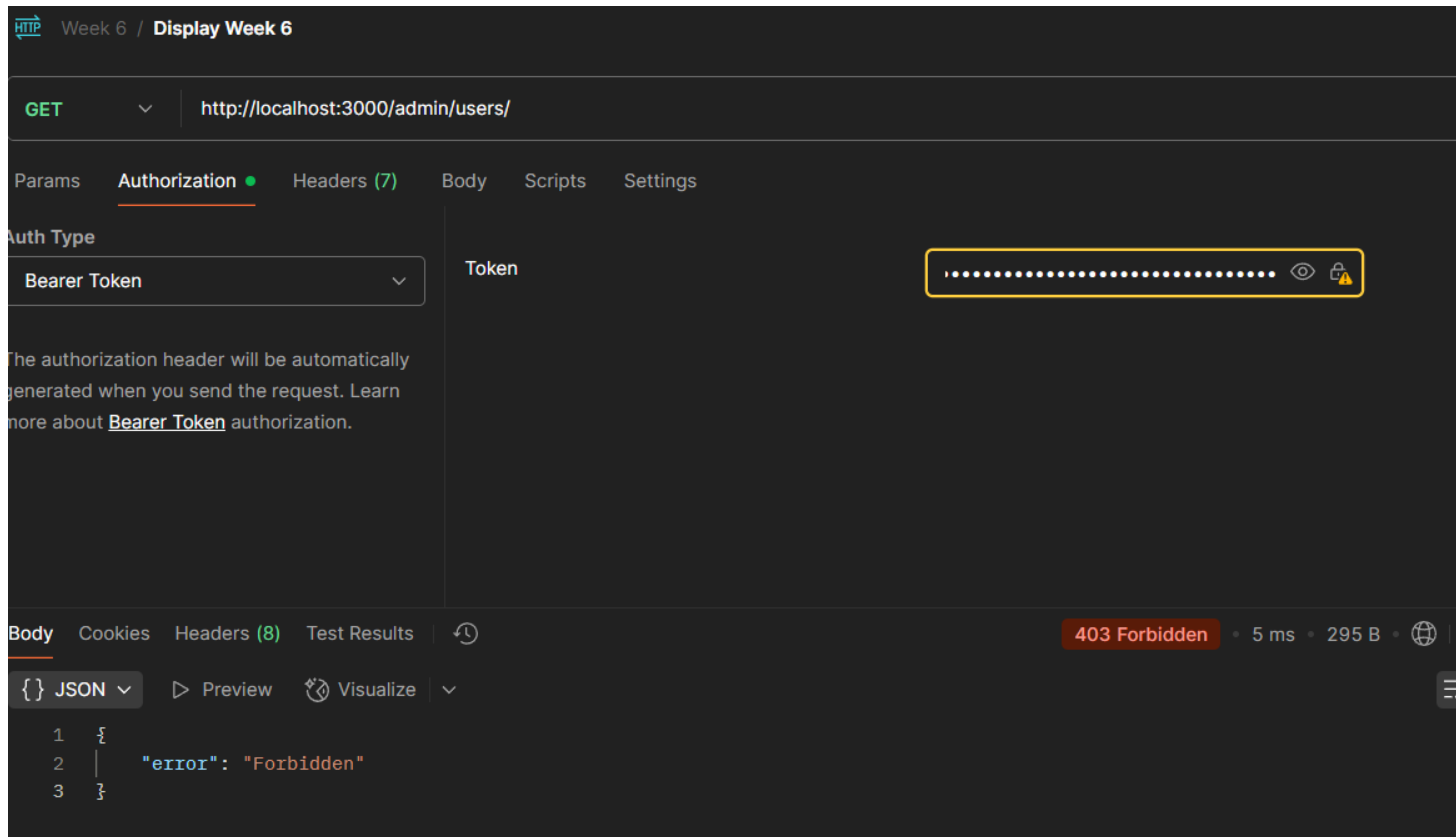
{ } JSON Preview Visualize

```
[{"_id": "6840f5b6d9f3ad4deb037c5d", "email": "fabrizioromano@gmail.com", "password": "$2b$10$KD1IEkddNauucmv0veTNxu7sRFY.Sn4cs2K7vieoiPCpfNqsZkTGe", "role": "driver"}, {"_id": "684254c638305bb009eca344", "email": "syafiq_iman@gmail.com", "password": "$2b$10$9ZlD8tDowXi0MDAV1q8bP.5s0GreMPXkY0ukFek0Qt/6Jpi6ZmKuq", "role": "admin"}]
```

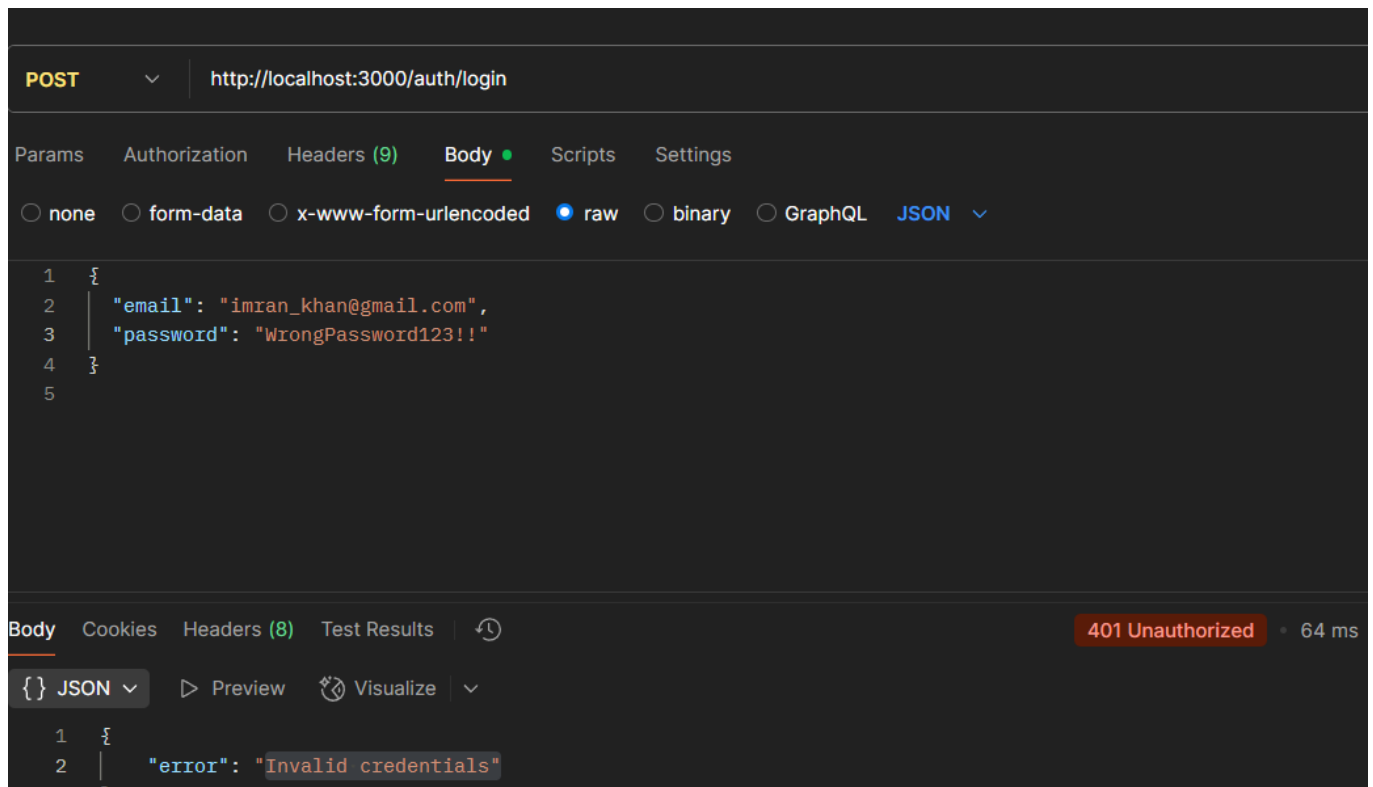
Successful Display of All Authorized Users



Successful Deleting User



Unsuccessful Display of All Users



Unsuccessful Log In because of Wrong Password

Link to GitHub Repository:

<https://github.com/Ahmad-Azeem/min-iman/tree/week6>